Programming with Generative AI

Viraj Kumar
Kotak-IISc AI-ML Centre

Lecture 8:

- Asking Clarifying Questions
- Strings



A CSR Initiative by



NPTEL

Task 1: num_digits

- Our client asks us to complete this function:

```
>>> num_digits(100)
3
```

```
>>> num_digits(100)
2
```



```
1    def num_digits(n: int) -> int:
2        """Count the number of digits in n.
3        >>> num_digits(37)
4        2
5        """
6        pass
7
```

CodeCheck

- Real-world tasks can be **ambiguous**
- It is **our** job to *ask clarifying questions*

Refute Our Friend's Function

```
1  def num_digits(n: int) -> int:
2      """Count the number of digits in n.
3      >>> num_digits(37)
4      2
5      >>> num_digits(100)
6      3
7      """
8      if -9 <= n <= 9:
9          return 1
10     return 1 + num_digits(n // 10)
```

Base case

Recursive case

Let's **trace** the code:

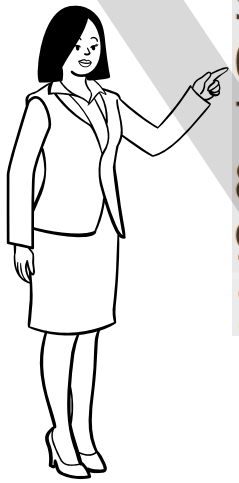
```
num_digits(234)
= 1 + num_digits(   )
= 1 + (1 + num_digits( ))
= 1 + (1 + 1) = 3
```

PythonTutor CodeCheck

NPTEL

Task 2: num_vowels

- Our client asks us to complete this function:



```
1    def num_vowels(s: str) -> int:
2        """Count the number of vowels in s.
3        >>> num_vowels('cat')
4        1
5        """
6        pass
7
8    # First check what the expected result
9    # should be on input(s) of your choice.
10   s = 'cat'
```

CodeCheck

NPTEL

Let's explore the `str` datatype

- This program asks for a name and it prints some information about that name:

```
1  name = input('Name: ')
2  length = len(name)
3  print(f'Length: {length}')
4  print(f'First letter: {name[0]}')
5  print(f'Last letter: {name[length - 1]}')
6
```

```
Name: Jasminder
Length: 9
First letter: J
Last letter: r
```

name	'J'	'a'	's'	'm'	'i'	'n'	'd'	'e'	'r'
Index	0	1	2	3	4	5	6	7	8

Test Your Understanding

- Given a string of length n, a **valid index** i satisfies the condition:

A) $-n \leq i < n$

B) $-n \leq i \leq n$

C) $-n < i < n$

D) $-n < i \leq n$

```
1  name = input('Name: ')
2  length = len(name)
3  print(f'Length: {length}')
4  print(f'First letter: {name[0]}')
```

```
Name:
Length: 0
```

name	'J'	'a'	's'	'm'	'i'	'n'	'd'	'e'	'r'
Index	0	1	2	3	4	5	6	7	8
Negative index	-9	-8	-7	-6	-5	-4	-3	-2	-1

Many powerful string **operators** and **methods**

- The `in` operator: Check if one string is a **substring** of another

```
>>> s = 'cat' 'banana'
>>> s[1:2]
    'a'          n 'banana'
>>> s[1:]
    'at'         n 'banana'
>>> s[:2]
    'ca'         ower() in 'banana'
>>> s[:-1]
    'ca'
```

The `lower()` **method** of the `str` object `'Nana'`

- String **slicing**: Make a copy of a substring of a given string

NPTEL

Test Your Understanding

- Which of these is an appropriate name for this function?

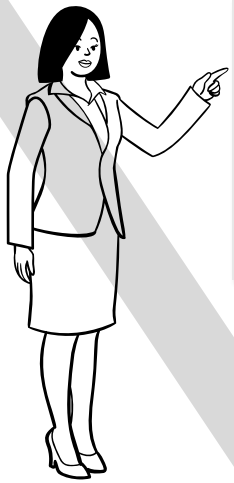
```
1   def mystery(s: str) -> int:
2       s = s.lower()
3       result = 0
4       if 'a' in s:
5           result = result + 1
6       if 'e' in s:
7           result += 1
8       if 'i' in s:
9           result += 1
10      if 'o' in s:
11          result += 1
12      if 'u' in s:
13          result += 1
14      return result
```

A) num_vowels
B) unique_vowels
C) lowercase_vowels
D) unique_lowercase_vowels

NPTEL

Let's solve Task 2!

- Our client asks us to complete this function:



```
    def num_vowels(s: str) -> int:
        """Count the number of vowels in s.
        >>> num_vowels('cat')
        1
        """
```

lowercase

```
if -9 <= n <= 9:
    return 1
return 1 + num_digits(n // 10)
```

```
num_vowels('cat')
= 0 + num_vowels('at')  # 'cat'[1:]
= 0 + (1 + num_vowels('t'))
```

Refute Our Friend's Function

```
1  def num_vowels(s: str) -> int:
2      """Count the number of lowercase vowels in s.
3      >>> num_vowels('cat')
4      1
5      >>> num_vowels('India')
6      2
7      """
8      first_vowel = 1 if s[0] in 'aeiou' else 0
9      if len(s) == 1:
10         return first_vowel
11     return first_vowel + num_vowels(s[1:])
12
13 print(num_vowels(''))
```

Edit this code

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (4 steps)

```
IndexError: string index out of range
```

NPTEL

Immutable Objects

- Objects of all types t
  (int, float, str) ar

```
TypeError: 'str' o
```

- We **cannot modify** a
  - We can assign **s** to a

```
>>> s = 'cat'
>>> s = 'bat'
>>>
```

```
>>> i = 37
>>> id(i)
140720580388904
>>> i += 1
>>> id(i)
140720580388936
>>> f = i / 2
>>> id(f)
1678048868432
>>> f *= 2
>>> id(f)
1678054097072
```

```
>>> s = 'cat'
>>> s[0] = 'b'
rt item assignment
```

NPTEL

Summary

- Real-world problems can be ambiguous
  - Our client is typically **not** a programmer
  - ***We*** must learn how to **ask the client clarifying questions**
  - We must **imagine** different ways in which the given task can be interpreted
- A powerful built-in datatype: `str`

  Immutable sequences (can be **indexed**)
  - Negative indexing
  - Slicing
  - Many operators and methods: `in`, `.lower()`, …

NPTEL